14 Integrating JPA and Hibernate with Spring

This chapter covers

- Introducing Spring Framework and dependency injection
- Examining the data access object (DAO) design pattern
- Creating and generifying a Spring JPA application using the DAO design pattern
- Creating and generifying a Spring Hibernate application using the DAO design pattern

In this chapter, we'll analyze a few different possibilities for integrating Spring and Hibernate. Spring is a lightweight but also flexible and universal Java framework. It is open source, and it can be used at the level of any layer in a Java application. We'll investigate the principles behind the Spring Framework (*dependency injection*, also known as *inversion of control*), and we'll use Spring together with JPA or Hibernate to build Java persistence applications.

NOTE To execute the examples from the source code, you'll first need to run the Ch14.sql script.

14.1 Spring Framework and dependency injection

Spring Framework provides a comprehensive infrastructure for developing Java applications. It handles the infrastructure so that you can focus on your application, and it enables you to build applications from plain old Java objects (POJOs).

Rod Johnson created Spring in 2002, beginning with his book *Expert One-on-One J2EE Design and Development* (Johnson, 2002). The basic idea behind Spring is that it simplifies the traditional approach to designing enterprise applications. For a quick introduction to the capabilities of the Spring Framework, the book *Spring Start Here* by Laurenţiu Spilcă (Spilcă, 2021) is a good source.

A Java application typically consists of objects that collaborate to solve a problem. The objects in a program depend on each other. You can use design patterns (factory, builder, proxy, decorator, and so on) to compose classes and objects, but this burden is on the side of the developer.

Spring implements various design patterns. Spring Framework's *dependency injection* pattern (also known as *inversion of control*, or IoC) supports the creation of an application consisting of different components and objects.

The key characteristic of a framework is exactly this dependency injection or IoC. When you call a method from JDK or a library, you are in control. In contrast, with a framework, control is inverted: the framework calls *you* (see figure 14.1). You must follow the paradigm provided by the framework and fill out your own code. The framework defines a skeleton, and you insert the features to fill out that skeleton. Your code is under the control of the framework, and the framework calls it. This way, you can focus on implementing the business logic rather than on the design.

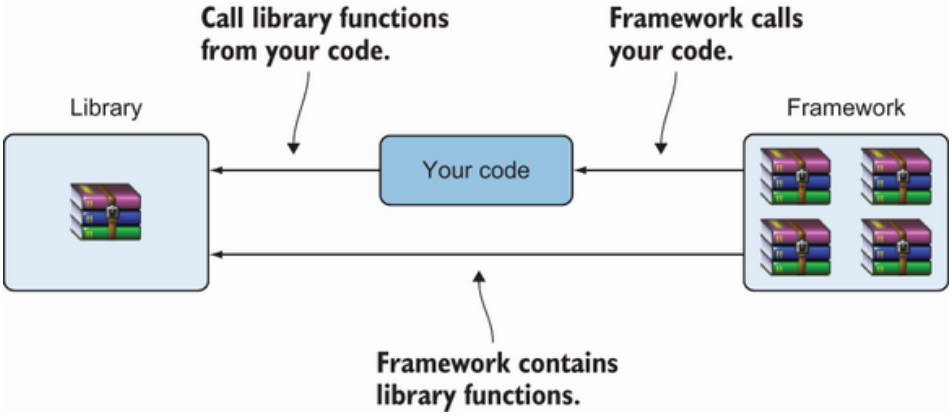

Figure 14.1 Your code calls a library. A framework calls your code.

The creation, dependency injection, and general lifecycle of objects under the control of the Spring Framework are managed by a container. The container will combine the application classes and the configuration information (metadata) to get a ready-to-run application (figure 14.2). The container is thus at the core of the IoC principle.

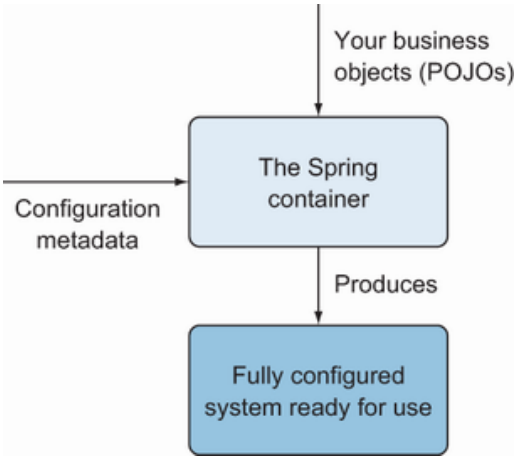

Figure 14.2 The functionality of the Spring IoC container

The objects under the management of the IoC container are called *beans*. The beans form the backbone of the Spring application.

14.2 JPA application using Spring and the DAO pattern

In this section, we'll look at how we can build a JPA application using Spring and the data access object (DAO) design pattern. The DAO design pattern creates an abstract interface to a database, supporting access operations without exposing any internals of the database.

You may argue that the Spring Data JPA repositories we have created and worked with already do this, and that is true. We'll demonstrate in this chapter how to build a DAO class, and we'll discuss when we should prefer this approach instead of using Spring Data JPA.

The CaveatEmptor application contains the `Item` and `Bid` classes (listings 14.1 and 14.2). The entities will now be managed with the help of the Spring Framework. The relationship between the `BID` and `ITEM` tables will be kept through a foreign key field on the `BID` table side. A field marked with the `@javax.persistence.Transient` annotation is excluded from persistence.

Listing 14.1 The `Item` class

```
Path: Ch14/spring-jpa-dao/src/main/java/com/manning/javapersistence/ch14
➥ /Item.java

@Entity
\1 Item {

    @Id                                                                    Ⓐ
    @GeneratedValue(generator = "ID_GENERATOR")                            Ⓐ
    private Long id;                                                       Ⓐ

    @NotNull                                                               Ⓑ
    @Size(                                                                 Ⓑ
            min = 2,                                                       Ⓑ
            max = 255,                                                     Ⓑ
            message = "Name is required, maximum 255 characters."          Ⓑ
    )                                                                      Ⓑ
    private String name;                                                   Ⓑ

    @Transient                                                             Ⓒ
    private Set<Bid> bids = new HashSet<>();                               Ⓒ

    // . . .
}
```

Ⓐ The `id` field is a generated identifier.

Ⓑ The `name` field is not null and must have a size from 2 to 255 characters.

Ⓒ Each `Item` has a reference to its set of `Bid`s. The field is marked as `@Transient`, so it is excluded from persistence.

We'll move our attention to the `Bid` class, as it looks now. It is also an entity, and the relationship between `Item` and `Bid` is one-to-many.

Listing 14.2 The `Bid` class

```
Path: Ch14/spring-jpa-dao/src/main/java/com/manning/javapersistence/ch14
➥ /Bid.java

@Entity
public class Bid {

    @Id                                                          Ⓐ
    @GeneratedValue(generator = "ID_GENERATOR")                  Ⓐ
    private Long id;                                             Ⓐ

    @NotNull                                                     Ⓑ
    private BigDecimal amount;                                   Ⓑ

    @ManyToOne(optional = false, fetch = FetchType.LAZY)         Ⓒ
    @JoinColumn(name = "ITEM_ID")                                Ⓒ
    private Item item;                                           Ⓒ
    // . . .
}
```

Ⓐ The `Bid` entity class contains the `id` field as the generated identifier.

Ⓑ The `amount` field should not be null.

Ⓒ Each `Bid` has a non-optional reference to its `Item`. The fetching will be lazy, and the name of the joining column is `ITEM_ID`.

To implement the DAO design pattern, we'll start by creating two interfaces, `ItemDao` and `BidDao`, and we'll declare the access operations that will be implemented:

```
Path: Ch14/spring-jpa-dao/src/main/java/com/manning/javapersistence/ch14
➥ /dao/ItemDao.java

public interface ItemDao {
    Item getById(long id);

    List<Item> getAll();

    void insert(Item item);

    void update(long id, String name);

    void delete(Item item);
```

```
    Item findByName(String name);
}
```

The `BidDao` interface is declared as follows:

```
Path: Ch14/spring-jpa-dao/src/main/java/com/manning/javapersistence/ch14
➥ /dao/BidDao.java

public interface BidDao {
    Bid getById(long id);

    List<Bid> getAll();

    void insert(Bid bid);

    void update(long id, String amount);

    void delete(Bid bid);

    List<Bid> findByAmount(String amount);
}
```

`@Repository` is a marker annotation indicating that the component represents a DAO. Besides marking the annotated class as a Spring component, `@Repository` will catch the persistence-specific exceptions and will translate them to Spring unchecked exceptions. `@Transactional` will make all the methods from inside the class transactional, as discussed in section 11.4.3.

An `EntityManager` is not thread-safe by itself. We'll use `@PersistenceContext` so that the container injects a proxy object that is thread-safe. Besides injecting the dependency on a container-managed entity manager, the `@PersistenceContext` annotation has parameters. Setting the persistence type to `EXTENDED` keeps the persistence context for the whole lifecycle of a bean.

The implementation of the `ItemDao` interface, `ItemDaoImpl`, is shown in the following listing.

Listing 14.3 The `ItemDaoImpl` class

```
Path: Ch14/spring-jpa-dao/src/main/java/com/manning/javapersistence/ch14
➥ /dao/ItemDaoImpl.java

@Repository
@Transactional
public class ItemDaoImpl implements ItemDao {
    @PersistenceContext(type = PersistenceContextType.EXTENDED)
```

```
    private EntityManager em;

    @Override
    public Item getById(long id) {
        return em.find(Item.class, id);
    }

    @Override
    public List<Item> getAll() {
        return (List<Item>) em.createQuery("from Item", Item.class)
                              .getResultList();
    }

    @Override
    public void insert(Item item) {
        em.persist(item);
        for (Bid bid : item.getBids()) {
            em.persist(bid);
        }
    }

    @Override
    public void update(long id, String name) {
        Item item = em.find(Item.class, id);
        item.setName(name);
        em.persist(item);
    }

    @Override
    public void delete(Item item) {
        for (Bid bid : item.getBids()) {
            em.remove(bid);
        }
        em.remove(item);
    }

    @Override
    public Item findByName(String name) {
        return em.createQuery("from Item where name=:name", Item.class)
                .setParameter("name", name).getSingleResult();
    }
}
```

Ⓐ The `ItemDaoImpl` class is annotated with `@Repository` and `@Transactional`.

Ⓑ The `EntityManager em` field is injected in the application, as annotated with `@PersistenceContext`. Persistence type `EXTENDED` means the persistence context is kept for the whole lifecycle of a bean.

Ⓒ Retrieve an `Item` by its `id`.

Ⓓ Retrieve all `Item` entities.

Ⓔ Persist an `Item` and all its `Bid`s.

Ⓕ Update the `name` field of an `Item`.

Ⓖ Remove all the bids belonging to an `Item` and the `Item` itself.

Ⓗ Search for an `Item` by its `name`.

The implementation of the `BidDao` interface, `BidDaoImpl`, is shown in the next listing.

Listing 14.4 The `BidDaoImpl` class

```
Path: Ch14/spring-jpa-dao/src/main/java/com/manning/javapersistence/ch14
➥ /dao/BidDaoImpl.java

@Repository
@Transactional
public class BidDaoImpl implements BidDao {
    @PersistenceContext(type = PersistenceContextType.EXTENDED)
    private EntityManager em;

    @Override
    public Bid getById(long id) {
        return em.find(Bid.class, id);
    }

    @Override
    public List<Bid> getAll() {
        return em.createQuery("from Bid", Bid.class).getResultList();
    }

    @Override
    public void insert(Bid bid) {
        em.persist(bid);
    }

    @Override
    public void update(long id, String amount) {
        Bid bid = em.find(Bid.class, id);
        bid.setAmount(new BigDecimal(amount));
        em.persist(bid);
    }

    @Override
    public void delete(Bid bid) {
        em.remove(bid);
    }

    @Override
    public List<Bid> findByAmount(String amount) {
        return em.createQuery("from Bid where amount=:amount", Bid.class
          .setParameter("amount", new BigDecimal(amount)).getResultList(
```

```
    }
}
```

Ⓐ The `BidDaoImpl` class is annotated with `@Repository` and `@Transactional`.

Ⓑ The `EntityManager em` field is injected in the application, as it is annotated with `@PersistenceContext`. Setting the persistence type to `EXTENDED` keeps the persistence context for the whole lifecycle of a bean.

Ⓒ Retrieve a `Bid` by its `id`.

Ⓓ Retrieve all `Bid` entities.

Ⓔ Persist a `Bid`.

Ⓕ Update the `amount` field of a `Bid`.

Ⓖ Remove a `Bid`.

Ⓗ Search for a `Bid` by its `amount`.

To work with the database, we'll provide a special class, `DatabaseService`, that will be responsible for populating the database and for removing the information from it.

Listing 14.5 The `DatabaseService` class

```
Path: Ch14/spring-jpa-dao/src/test/java/com/manning/javapersistence/ch14
➥ /DatabaseService.java

public class DatabaseService {

    @PersistenceContext(type = PersistenceContextType.EXTENDED)        Ⓐ
    private EntityManager em;                                          Ⓐ

    @Autowired                                                         Ⓑ
    private ItemDao itemDao;                                           Ⓑ

    @Transactional                                                     Ⓒ
    public void init() {                                               Ⓒ
        for (int i = 0; i < 10; i++) {                                 Ⓒ
            String itemName = "Item " + (i + 1);                       Ⓒ
            Item item = new Item();                                    Ⓒ
            item.setName(itemName);                                    Ⓒ
            Bid bid1 = new Bid(new BigDecimal(1000.0), item);          Ⓒ
            Bid bid2 = new Bid(new BigDecimal(1100.0), item);          Ⓒ

            itemDao.insert(item);                                      Ⓒ
        }                                                              Ⓒ
```

```
    }

    @Transactional                                                    Ⓓ
    public void clear() {                                             Ⓓ
        em.createQuery("delete from Bid b").executeUpdate();          Ⓓ
        em.createQuery("delete from Item i").executeUpdate();         Ⓓ
    }

}
```

Ⓐ The `EntityManager em` field is injected in the application, as it is annotated with `@PersistenceContext`. Setting the persistence type to `EXTENDED` keeps the persistence context for the whole lifecycle of a bean.

Ⓑ The `ItemDao itemDao` field is injected in the application, as it is annotated with `@Autowired`. Because the `ItemDaoImpl` class is annotated with `@Repository`, Spring will create the needed bean belonging to this class, to be injected here.

Ⓒ Generate 10 `Item` objects, each of them having 2 `Bid`s, and insert them in the database.

Ⓓ Delete all previously inserted `Bid` and `Item` objects.

The standard configuration file for Spring is a Java class that creates and sets up the needed beans. The `@EnableTransactionManagement` annotation will enable Spring's annotation-driven transaction management capability. When using XML configuration, this annotation is mirrored by the `tx:annotation-driven` element. Every interaction with the database should occur within transaction boundaries and Spring needs a transaction manager bean.

We'll create the following configuration file for the application.

Listing 14.6 The `SpringConfiguration` class

```
Path: Ch14/spring-jpa-dao/src/test/java/com/manning/javapersistence/ch14
➥ /configuration/SpringConfiguration.java

@EnableTransactionManagement
public class SpringConfiguration {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl(
            "jdbc:mysql://localhost:3306/CH14_SPRING_HIBERNATE
        ➥ ?serverTimezone=UTC");                                     Ⓓ
        dataSource.setUsername("root");
        dataSource.setPassword("");
```

```
        return dataSource;
    }

    @Bean
    public DatabaseService databaseService() {
        return new DatabaseService();
    }

    @Bean
    public JpaTransactionManager
           transactionManager(EntityManagerFactory emf){
        return new JpaTransactionManager(emf);
    }


    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory()
        LocalContainerEntityManagerFactoryBean
        localContainerEntityManagerFactoryBean =
                new LocalContainerEntityManagerFactoryBean();
        localContainerEntityManagerFactoryBean
               .setPersistenceUnitName("ch14");
        localContainerEntityManagerFactoryBean.setDataSource(dataSource(
        localContainerEntityManagerFactoryBean.setPackagesToScan(
            "com.manning.javapersistence.ch14");
        return localContainerEntityManagerFactoryBean;
    }

    @Bean
    public ItemDao itemDao() {
        return new ItemDaoImpl();
    }

    @Bean
    public BidDao bidDao() {
        return new BidDaoImpl();
    }
}
```

Ⓐ The `@EnableTransactionManagement` annotation enables Spring's annotation-driven transaction management capability.

Ⓑ Create a data source bean.

Ⓒ Indicate the JDBC properties—the driver.

Ⓓ The URL of the database.

Ⓔ The username.

Ⓕ There is no password in this configuration. Modify the credentials to correspond to the ones on your machine and use a password in practice.

Ⓖ The `DatabaseService` bean that Spring will use to populate and clear the database.

Ⓗ Create a transaction manager bean based on an entity manager factory.

Ⓘ `LocalContainerEntityManagerFactoryBean` is a factory bean that produces an `EntityManagerFactory` following the JPA standard container bootstrap contract.

Ⓙ Set the persistence unit name, defined in persistence.xml.

Ⓚ Set the data source.

Ⓛ Set the packages to scan for entity classes. Beans are located in `com.manning.javapersistence.ch14`, so we set this package to be scanned.

Ⓜ Create an `ItemDao` bean.

Ⓝ Create a `BidDao` bean.

This configuration information is used by Spring to create and inject the beans that form the backbone of the application. We can use the alternative XML configuration, and the application-context.xml file mirrors the work done in SpringConfiguration .java. We would just like to emphasize one thing we mentioned previously: in XML, we enable Spring's annotation-driven transaction management capability with the help of the `tx:annotation-driven` element, referencing a transaction manager bean:

```
Path: Ch14/spring-jpa-dao/src/test/resources/application-context.xml

<tx:annotation-driven transaction-manager="txManager"/>
```

The `SpringExtension` extension is used to integrate the Spring test context with the JUnit 5 Jupiter test by implementing several JUnit Jupiter extension model callback methods.

It is important to use the type `PersistenceContextType.` *EXTENDED* for all injected `EntityManager` beans. If we used the default `PersistenceContextType.TRANSACTION` type, the returned object would become detached at the end of a transaction's execution. Passing it to the `delete` method would result in an "IllegalArgumentException: Removing a detached instance" exception.

It is time to test the functionality we've developed for persisting the `Item` and `Bid` entities.

Listing 14.7 The `SpringJpaTest` class

```
Path: Ch14/spring-jpa-dao/src/test/java/com/manning/javapersistence/ch14
➥ /SpringJpaTest.java

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringConfiguration.class})
//@ContextConfiguration("classpath:application-context.xml")
public class SpringJpaTest {

    @Autowired
    private DatabaseService databaseService;

    @Autowired
    private ItemDao itemDao;

    @Autowired
    private BidDao bidDao;

    @BeforeEach
    public void setUp() {
        databaseService.init();
    }

    @Test
    public void testInsertItems() {
        List<Item> itemsList = itemDao.getAll();
        List<Bid> bidsList = bidDao.getAll();
        assertAll(
                () -> assertNotNull(itemsList),
                () -> assertEquals(10, itemsList.size()),
                () -> assertNotNull(itemDao.findByName("Item 1")),
                () -> assertNotNull(bidsList),
                () -> assertEquals(20, bidsList.size()),
                () -> assertEquals(10,
                                bidDao.findByAmount("1000.00").size()
        );
    }

    @Test
    public void testDeleteItem() {
        itemDao.delete(itemDao.findByName("Item 2"));
        assertThrows(NoResultException.class,
                    () -> itemDao.findByName("Item 2"));
    }


    // . . .

    @AfterEach
    public void dropDown() {
        databaseService.clear();
    }
```

```
}
```

Ⓐ Extend the test using `SpringExtension`. As previously mentioned, this will integrate the Spring TestContext Framework into JUnit 5 by implementing several JUnit Jupiter extension model callback methods.

Ⓑ The Spring test context is configured using the beans defined in the previously presented `SpringConfiguration` class.

Ⓒ Alternatively, we can configure the test context using XML. Only one Ⓑ of the or Ⓒ lines should be active in the code.

Ⓓ Autowire one `DatabaseService` bean, one `ItemDao` bean, and one `BidDao` bean.

Ⓔ Before the execution of each test, the content of the database is initialized by the `init` method from the injected `DatabaseService`.

Ⓕ Retrieve all `Item`s and all `Bid`s and do verifications.

Ⓖ Find an `Item` by its `name` field and delete it from the database. We will use `PersistenceContextType.EXTENDED` for all injected `EntityManager` beans. Otherwise, passing it to the `delete` method would result in an "IllegalArgument-Exception: Removing a detached instance" exception.

Ⓗ After the successful deletion of the `Item` from the database, trying to find it again will throw a `NoResultException`. The rest of the tests can be easily investigated in the source code.

Ⓘ After the execution of each test, the content of the database is erased by the `clear` method from the injected `DatabaseService`.

When should we apply such a solution using the Spring Framework and the DAO design pattern? There are a few situations when we would recommend it:

- You would like to hand off the task of controlling the entity manager and the transactions to the Spring Framework (remember that this is done through the *inversion of control*). The tradeoff is that you lose the possibility of debugging the transactions. Just be aware of that.

- You would like to create your own API for managing persistence and either you cannot or you do not want to use Spring Data. This might happen when you have very particular operations that you need to control, or you want to remove the Spring Data overhead (including the ramp-up time for the team to adopt it, introducing new dependencies in an already existing project, and the execution delay of Spring Data, as was discussed in section 2.7).
- In particular situations, you may want to handle the entity manager and the transactions to the Spring Framework and still not implement your own DAO classes.

We would like to improve on the design of our Spring persistence application. The next sections are dedicated to making it more generic and using the Hibernate API instead of JPA. We'll focus on the differences between this first solution and our new versions, and we'll discuss how to introduce the changes.

14.3 Generifying a JPA application that uses Spring and DAO

If we take a closer look at the `ItemDao` and `BidDao` interfaces and `ItemDaoImpl` and `BidDaoImpl` classes we've created, we'll find a few shortcomings:

- There are similar operations, such as `getById`, `getAll`, `insert`, and `delete`, that differ mainly in the type of argument they receive or in the returned result.
- The `update` method receives as a second argument the value of a particular property. We may need to write multiple methods if we need to update different properties of an entity.
- Methods such as `findByName` or `findByAmount` are tied to particular properties. We may need to write different methods for finding an entity using different properties.

Consequently, we'll introduce a `GenericDao` interface.

Listing 14.8 The `GenericDao` interface

```
Path: Ch14/spring-jpa-dao-gen/src/main/java/com/manning/javapersistence
➥ /ch14/dao/GenericDao.java

public interface GenericDao<T> {
   T getById(long id);

   List<T> getAll();

   void insert(T entity);

   void delete(T entity);
```

```
    void update(long id, String propertyName, Object propertyValue);

    List<T> findByProperty(String propertyName, Object propertyValue);

}
```

Ⓐ The `getById` and `getAll` methods have a generic return type.

Ⓑ The `insert` and `update` methods have generic input, `T entity`.

Ⓒ The `update` and `findByProperty` methods will receive as arguments the `propertyName` and the new `propertyValue`.

We'll create an abstract implementation of the `GenericDao` interface, called `AbstractGenericDao`, as shown in listing 14.9. Here we'll write the common functionality of all DAO classes, and we'll let concrete classes implement their specifics.

We'll inject an `EntityManager em` field in the application, annotating it with `@PersistenceContext`. Setting the persistence type to `EXTENDED` keeps the persistence context for the whole lifecycle of a bean.

Listing 14.9 The `AbstractGenericDao` class

```
Path: Ch14/spring-jpa-dao-gen/src/main/java/com/manning/javapersistence
➥ /ch14/dao/AbstractGenericDao.java

@Repository
@Transactional
public abstract class AbstractGenericDao<T> implements GenericDao<T> {

    @PersistenceContext(type = PersistenceContextType.EXTENDED)
    protected EntityManager em;
    private Class<T> clazz;

    public void setClazz(Class<T> clazz) {
        this.clazz = clazz;
    }

    @Override
    public T getById(long id) {
        return em.createQuery(
                "SELECT e FROM " + clazz.getName() + " e WHERE e.id = :i
                 clazz).setParameter("id", id).getSingleResult();
    }

    @Override
    public List<T> getAll() {
        return em.createQuery("from " +
                              clazz.getName(), clazz).getResultList();
```

```
    }

    @Override
    public void insert(T entity) {
        em.persist(entity);
    }

    @Override
    public void delete(T entity) {
        em.remove(entity);
    }

    @Override
    public void update(long id, String propertyName, Object propertyValu
        em.createQuery("UPDATE " + clazz.getName() + " e SET e." +
                 propertyName + " = :propertyValue WHERE e.id = :id")
                .setParameter("propertyValue", propertyValue)
                .setParameter("id", id).executeUpdate();
    }

    @Override
    public List<T> findByProperty(String propertyName,
                                  Object propertyValue) {
        return em.createQuery(
               "SELECT e FROM " + clazz.getName() + " e WHERE e." +
                propertyName + " = :propertyValue", clazz)
                .setParameter("propertyValue", propertyValue)
                .getResultList();
    }
}
```

Ⓐ The `AbstractGenericDao` class is annotated with `@Repository` and `@Transactional`.

Ⓑ The `EXTENDED` persistence type of the `EntityManager` will keep the persistence context for the whole lifecycle of a bean. The field is `protected`, to be eventually inherited and used by subclasses.

Ⓒ `clazz` is the effective `Class` field on which the DAO will work.

Ⓓ Execute a `SELECT` query using the `clazz` entity and setting the `id` as a parameter.

Ⓔ Execute a `SELECT` query using the `clazz` entity and get the list of the results.

Ⓕ Persist the `entity`.

Ⓖ Remove the `entity`.

Ⓗ Execute an `UPDATE` using the `propertyName`, the `propertyValue`, and the `id`.

Ⓘ Execute a `SELECT` using the `propertyName` and the `propertyValue`.

The `AbstractGenericDao` class provides most of the general DAO functionality. It only needs to be customized a little for particular DAO classes. The `ItemDaoImpl` class will extend the `AbstractGenericDao` class and will override some of the methods.

Listing 14.10 The `ItemDaoImpl` class extending AbstractGenericDao

```
Path: Ch14/spring-jpa-dao-gen/src/main/java/com/manning/javapersistence
➥ /ch14/dao/ItemDaoImpl.java

public class ItemDaoImpl extends AbstractGenericDao<Item> {          Ⓐ

    public ItemDaoImpl() {                                           Ⓑ
        setClazz(Item.class);                                        Ⓑ
    }

    @Override                                                        Ⓒ
    public void insert(Item item) {                                  Ⓒ
        em.persist(item);                                            Ⓒ
        for (Bid bid : item.getBids()) {                             Ⓒ
            em.persist(bid);                                         Ⓒ
        }                                                            Ⓒ
    }

    @Override                                                        Ⓓ
    public void delete(Item item) {                                  Ⓓ
        for (Bid bid: item.getBids()) {                              Ⓓ
            em.remove(bid);                                          Ⓓ
        }                                                            Ⓓ
        em.remove(item);                                             Ⓓ
    }

}
```

Ⓐ `ItemDaoImpl` extends `AbstractGenericDao` and is generified by `Item`.

Ⓑ The constructor sets `Item.class` as the entity class to be managed.

Ⓒ Persist the `Item` entity and all its `Bid` entities. The `EntityManager em` field is inherited from the `AbstractGenericDao` class.

Ⓓ Remove all the bids belonging to an `Item` and the `Item` itself.

The `BidDaoImpl` class will simply extend the `AbstractGenericDao` class and set the entity class to manage.

Listing 14.11 The `BidDaoImpl` class extending AbstractGenericDao

```
Path: Ch14/spring-jpa-dao-gen/src/main/java/com/manning/javapersistence
➥ /ch14/dao/BidDaoImpl.java

public class BidDaoImpl extends AbstractGenericDao<Bid> {      Ⓐ

    public BidDaoImpl() {                                      Ⓑ
        setClazz(Bid.class);                                   Ⓑ
    }

}
```

Ⓐ `BidDaoImpl` extends `AbstractGenericDao` and is generified by `Bid`.

Ⓑ The constructor sets `Bid.class` as the entity class to be managed. All other methods are inherited from `AbstractGenericDao` and are fully reusable this way.

Some small changes are needed for the configuration and testing classes. The `SpringConfiguration` class will now declare the two DAO beans as `GenericDao`:

```
Path: Ch14/spring-jpa-dao-gen/src/test/java/com/manning/javapersistence
➥ /ch14/configuration/SpringConfiguration.java

@Bean
public GenericDao<Item> itemDao() {
    return new ItemDaoImpl();
}

@Bean
public GenericDao<Bid> bidDao() {
    return new BidDaoImpl();
}
```

The `DatabaseService` class will inject the `itemDao` field as `GenericDao`:

```
Path: Ch14/spring-jpa-dao-gen/src/test/java/com/manning/javapersistence
➥ /ch14/DatabaseService.java

@Autowired
private GenericDao<Item> itemDao;
```

The `SpringJpaTest` class will inject the `itemDao` and `bidDao` fields as `GenericDao`:

```
Path: Ch14/spring-jpa-dao-gen/src/test/java/com/manning/javapersistence
➥ /ch14/SpringJpaTest.java

@Autowired
private GenericDao<Item> itemDao;

@Autowired
private GenericDao<Bid> bidDao;
```

We have now developed an easily extensible DAO class hierarchy using the JPA API. We can reuse the already written generic functionality or we can quickly overwrite some methods for particular entities (as was the case for `ItemDaoImpl`).

Let's now move on to the alternative of implementing a Hibernate application using Spring and the DAO pattern.

14.4 Hibernate application using Spring and the DAO pattern

We'll now demonstrate how to use Spring and the DAO pattern with the Hibernate API. As we previously mentioned, we'll emphasize only the changes between this approach and the previous applications.

Calling `sessionFactory.getCurrentSession()` will create a new `Session`, if one does not exist. Otherwise, it will use the existing session from the Hibernate context. The session will be automatically flushed and closed when a transaction ends. Using `sessionFactory.getCurrentSession()` is ideal in single-threaded applications, as using a single session will boost performance. In a multithreaded application, the session is not thread-safe, so you should use `sessionFactory.openSession()` and explicitly close the opened session. Or, as `Session` implements `AutoCloseable`, it can be used in try-with-resources blocks.

As the `Item` and `Bid` classes and the `ItemDao` and `BidDao` interfaces remain unchanged, we'll move on to `ItemDaoImpl` and `BidDaoImpl` and see what they look like now.

Listing 14.12 The `ItemDaoImpl` class using the Hibernate API

```
Path: Ch14/spring-hibernate-dao/src/main/java/com/manning/javapersistenc
➥ /ch14/dao/ItemDaoImpl.java

@Repository
@Transactional
```

```
public class ItemDaoImpl implements ItemDao {
    @Autowired
    private SessionFactory sessionFactory;

    @Override
    public Item getById(long id) {
        return sessionFactory.getCurrentSession().get(Item.class, id);
    }

    @Override
    public List<Item> getAll() {
        return sessionFactory.getCurrentSession()
                .createQuery("from Item", Item.class).list();
    }

    @Override
    public void insert(Item item) {
        sessionFactory.getCurrentSession().persist(item);
        for (Bid bid : item.getBids()) {
            sessionFactory.getCurrentSession().persist(bid);
        }
    }

    @Override
    public void update(long id, String name) {
        Item item = sessionFactory.getCurrentSession().get(Item.class, i
        item.setName(name);
        sessionFactory.getCurrentSession().update(item);
    }

    @Override
    public void delete(Item item) {
        sessionFactory.getCurrentSession()
            .createQuery("delete from Bid b where b.item.id = :id")
            .setParameter("id", item.getId()).executeUpdate();
        sessionFactory.getCurrentSession()
            .createQuery("delete from Item i where i.id = :id")
            .setParameter("id", item.getId()).executeUpdate();
    }

    @Override
    public Item findByName(String name) {
        return sessionFactory.getCurrentSession()
                 .createQuery("from Item where name=:name", Item.class)
                 .setParameter("name", name).uniqueResult();
    }
}
```

Ⓐ The `ItemDaoImpl` class is annotated with `@Repository` and `@Transactional`.

Ⓑ The `SessionFactory sessionFactory` field is injected in the application, as it is annotated with `@Autowired`.

Ⓒ Retrieve an `Item` by its `id`. Calling `sessionFactory.getCurrentSession()` will create a new `Session` if one does not exist.

Ⓓ Retrieve all `Item` entities.

Ⓔ Persist an `Item` and all its `Bid`s.

Ⓕ Update the `name` field of an `Item`.

Ⓖ Remove all the bids belonging to an `Item` and the `Item` itself.

Ⓗ Search for an `Item` by its `name`.

The `BidDaoImpl` class will also mirror the previously implemented functionality that used JPA and `EntityManager`, but it will use the Hibernate API and `SessionFactory`.

There are also important changes in the `SpringConfiguration` class. Moving from JPA to Hibernate, the `EntityManagerFactory` bean to be injected will be replaced with a `SessionFactory`. Similarly, the `JpaTransactionManager` bean to be injected will be replaced with a `HibernateTransactionManager`.

Listing 14.13 The `SpringConfiguration` class using the Hibernate API

```
Path: Ch14/spring-hibernate-dao/src/test/java/com/manning/javapersistenc
➥ /ch14/configuration/SpringConfiguration.java

@EnableTransactionManagement
public class SpringConfiguration {

    @Bean
    public LocalSessionFactoryBean sessionFactory() {
        LocalSessionFactoryBean sessionFactory =
             new LocalSessionFactoryBean();
        sessionFactory.setDataSource(dataSource());
        sessionFactory.setPackagesToScan(
             new String[]{"com.manning.javapersistence.ch14"});
        sessionFactory.setHibernateProperties(hibernateProperties());

        return sessionFactory;
    }

    private Properties hibernateProperties() {
        Properties hibernateProperties = new Properties();
        hibernateProperties.setProperty(AvailableSettings.HBM2DDL_AUTO,
                                                          "create");
        hibernateProperties.setProperty(AvailableSettings.SHOW_SQL,
                                                          "true");
        hibernateProperties.setProperty(AvailableSettings.DIALECT,
                              "org.hibernate.dialect.MySQL8Dialect");
```

```
        return hibernateProperties;
    }

    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl(
       "jdbc:mysql://localhost:3306/CH14_SPRING_HIBERNATE?serverTimezone
        ➥ UTC ");
        dataSource.setUsername("root");
        dataSource.setPassword("");
        return dataSource;
    }

    @Bean
    public DatabaseService databaseService() {
        return new DatabaseService();
    }

    @Bean
    public HibernateTransactionManager transactionManager(
                                        SessionFactory sessionFactory) {
        HibernateTransactionManager transactionManager
            = new HibernateTransactionManager();
        transactionManager.setSessionFactory(sessionFactory);
        return transactionManager;
    }

    @Bean
    public ItemDao itemDao() {
        return new ItemDaoImpl();
    }

    @Bean
    public BidDao bidDao() {
        return new BidDaoImpl();
    }
}
```

Ⓐ The `@EnableTransactionManagement` annotation will enable Spring's annotation-driven transaction management capability.

Ⓑ A `LocalSessionFactoryBean` is the `sessionFactory` object to be injected.

Ⓒ Set the data source and the packages to scan.

Ⓓ Set the Hibernate properties that will be provided from a separate method.

Ⓔ Create the Hibernate properties in a separate method.

Ⓕ Create a data source bean.

Ⓖ Indicate the JDBC properties—the driver.

Ⓗ The URL of the database.

Ⓘ The username.

Ⓙ There is no password in this configuration. Modify the credentials to correspond to the ones on your machine, and use a password in practice.

Ⓚ A `DatabaseService` bean will be used by Spring to populate and clear the database.

Ⓛ Create a transaction manager bean based on a session factory. Every interaction with the database should occur within transaction boundaries, so Spring needs a transaction manager bean.

Ⓜ Create an `ItemDao` bean.

Ⓝ Create a `BidDao` bean.

We can use the alternative of XML configuration, and the application-context.xml file should mirror the work done in SpringConfiguration.java. Enabling Spring's annotation-driven transaction management capability is done with the help of the `tx:annotation-driven` element, referencing a transaction manager bean, instead of the `@EnableTransactionManagement` annotation.

We'll now demonstrate how to generify this application that uses the Hibernate API instead of JPA. As usual, we'll focus on the differences from the initial solution and how to introduce the changes.

14.5 Generifying a Hibernate application that uses Spring and DAO

Keep in mind the shortcomings that we previously identified for the JPA solution using Data Access Object:

- There are similar operations, such as `getById`, `getAll`, `insert`, and `delete`, that differ mainly in the type of argument they receive or in the returned result.
- The `update` method receives as a second argument the value of a particular property. We may need to write multiple methods if we need to update different properties of an entity.

- Methods such as `findByName` or `findByAmount` are tied to particular properties. We may need to write different methods for finding an entity using different properties.

To address these shortcomings, we introduced the `GenericDao` interface, shown earlier in listing 14.8. This interface was implemented by the `AbstractGenericDao` class (listing 14.9), which now needs to be rewritten using the Hibernate API.

Listing 14.14 The `AbstractGenericDao` class using the Hibernate API

```
Path: Ch14/spring-hibernate-dao-gen/src/main/java/com/manning
➥ /javapersistence/ch14/dao/AbstractGenericDao.java

@Repository
@Transactional
public abstract class AbstractGenericDao<T> implements GenericDao<T> {

    @Autowired
    protected SessionFactory sessionFactory;
    private Class<T> clazz;

    public void setClazz(Class<T> clazz) {
        this.clazz = clazz;
    }

    @Override
    public T getById(long id) {
        return sessionFactory.getCurrentSession()
           .createQuery("SELECT e FROM " + clazz.getName() +
                        " e WHERE e.id = :id", clazz)
           .setParameter("id", id).getSingleResult();
    }

    @Override
    public List<T> getAll() {
        return sessionFactory.getCurrentSession()
           .createQuery("from " + clazz.getName(), clazz).getResultList(
    }

    @Override
    public void insert(T entity) {
        sessionFactory.getCurrentSession().persist(entity);
    }

    @Override
    public void delete(T entity) {
        sessionFactory.getCurrentSession().delete(entity);
    }

    @Override
    public void update(long id, String propertyName, Object propertyValu
        sessionFactory.getCurrentSession()
```

```
            .createQuery("UPDATE " + clazz.getName() + " e SET e." +
                  propertyName + " = :propertyValue WHERE e.id = :id")
            .setParameter("propertyValue", propertyValue)
            .setParameter("id", id).executeUpdate();
    }

    @Override
    public List<T> findByProperty(String propertyName,
   Object propertyValue) {
        return sessionFactory.getCurrentSession()
       .createQuery("SELECT e FROM " + clazz.getName() + " e WHERE e." +
           propertyName + " = :propertyValue", clazz)
       .setParameter("propertyValue", propertyValue).getResultList();
    }
}
```

Ⓐ The `AbstractGenericDao` class is annotated with `@Repository` and `@Transactional`.

Ⓑ The `SessionFactory sessionFactory` field is injected in the application, as it is annotated with `@Autowired`. It is `protected`, to be inherited and used by subclasses.

Ⓒ `clazz` is the effective `Class` field on which the DAO will work.

Ⓓ Execute a `SELECT` query using the `clazz` entity and setting the `id` as a parameter.

Ⓔ Execute a `SELECT` query using the `clazz` entity and get the list of the results.

Ⓕ Persist the `entity`.

Ⓖ Remove the `entity`.

Ⓗ Execute an `UPDATE` using the `propertyName`, the `propertyValue`, and the `id`.

Ⓘ Execute a `SELECT` using the `propertyName` and the `propertyValue`.

We'll customize the `AbstractGenericDao` class when it is extended by `ItemDaoImpl` and `BidDaoImpl`, this time using the Hibernate API.

The `ItemDaoImpl` class will extend the `AbstractGenericDao` class and will override some of the methods.

Listing 14.15 The `ItemDaoImpl` class using the Hibernate API

```
Path: Ch14/spring-jpa-hibernate-gen/src/main/java/com/manning
➥ /javapersistence/ch14/dao/ItemDaoImpl.java

public class ItemDaoImpl extends AbstractGenericDao<Item> {

    public ItemDaoImpl() {
        setClazz(Item.class);
    }

    @Override
    public void insert(Item item) {
        sessionFactory.getCurrentSession().persist(item);
        for (Bid bid : item.getBids()) {
            sessionFactory.getCurrentSession().persist(bid);
        }
    }

    @Override
    public void delete(Item item) {
        sessionFactory.getCurrentSession()
           .createQuery("delete from Bid b where b.item.id = :id")
           .setParameter("id", item.getId()).executeUpdate();
        sessionFactory.getCurrentSession()
           .createQuery("delete from Item i where i.id = :id")
           .setParameter("id", item.getId()).executeUpdate();
    }

}
```

Ⓐ `ItemDaoImpl` extends `AbstractGenericDao` and is generified by `Item`.

Ⓑ The constructor sets `Item.class` as the entity class to be managed.

Ⓒ Persist the `Item` entity and all its `Bid` entities. The `sessionFactory` field is inherited from the `AbstractGenericDao` class.

Ⓓ Remove all the bids belonging to an `Item` and the `Item` itself.

The `BidDaoImpl` class will simply extend the `AbstractGenericDao` class and set the entity class to manage.

Listing 14.16 The `BidDaoImpl` class using the Hibernate API

```
Path: Ch14/spring-hibernate-dao-gen/src/main/java/com/manning
➥ /javapersistence/ch14/dao/BidDaoImpl.java

public class BidDaoImpl extends AbstractGenericDao<Bid> {     Ⓐ

    public BidDaoImpl() {                                     Ⓑ
        setClazz(Bid.class);                                  Ⓑ
```

```
    }

}
```

Ⓐ `BidDaoImpl` extends `AbstractGenericDao` and is generified by `Bid`.

Ⓑ The constructor sets `Bid.class` as the entity class to be managed. All other methods are inherited from `AbstractGenericDao` and are fully reusable this way.

We have developed an easily extensible DAO class hierarchy using the Hibernate API. We can reuse the already written generic functionality or we can quickly overwrite some methods for particular entities (as we did for `ItemDaoImpl`).

Summary

- The dependency injection design pattern is the base of the Spring Framework, supporting the creation of applications consisting of different components and objects.
- You can develop a JPA application using the Spring Framework and the DAO pattern. Spring controls the `EntityManager`, the transaction manager, and other beans used by the application.
- You can generify a JPA application to provide a generic and easily extensible DAO base class. The DAO base class contains the common behavior to be inherited by all derived DAOs and will allow them to implement only their particular behavior.
- You can develop a Hibernate application using the Spring Framework and the DAO pattern. Spring controls the `SessionFactory`, the transaction manager, and other beans used by the application.
- You can generify a Hibernate application to provide a generic and easily extensible DAO base class. The DAO base class contains the common behavior to be inherited by all derived DAOs and will allow them to implement only their particular behavior.